

BizOS -- FEATURE TEST DOCUMENT

Real Estate SaaS AI Business Operating System

Last updated: 2026-06-30

Run this document top-to-bottom to verify all system features.

PREREQUISITES

1. Copy .env.example to .env
2. Set ANTHROPIC_API_KEY in .env (required for all agent tests)
3. From bizos/ directory, run: streamlit run dashboard/app.py
4. Dashboard should open at http://localhost:8501

Optional (stub mode active if absent):

- AIRTABLE_API_KEY + AIRTABLE_BASE_ID ? real CRM
- SLACK_WEBHOOK_URL ? real Slack alerts
- APOLLO_API_KEY or HUNTER_IO_KEY ? real lead data
- BUFFER_ACCESS_TOKEN ? real social scheduling

SECTION 1: CORE INFRASTRUCTURE**TEST 1.1 -- Database initializes cleanly**

Steps:

Delete data/bizos.db if it exists

Run: `python -c "from orchestrator.state import init_db; init_db(); print('OK')"`

Expected: prints OK, file data/bizos.db created

TEST 1.2 -- Input validation rejects bad payloads

Steps:

`python -c "``from orchestrator.state import create_task``try:``create_task('lead_gen', 'test', 'not a dict')``except (ValueError, TypeError) as e:``print('BLOCKED:', e)``"`

Expected: prints BLOCKED with validation error

TEST 1.3 -- UUID validation on approval resolution

Steps:

`python -c "``from orchestrator.state import resolve_approval``try:``resolve_approval('not-a-uuid', True)``except ValueError as e:``print('BLOCKED:', e)``"`

Expected: prints BLOCKED: Invalid approval_id

TEST 1.4 -- Agent name validation

Steps:

`python -c "``from orchestrator.router import dispatch`

```
result = dispatch('hacker_agent', 'test', {})
print(result)
"
```

Expected: {'status': 'failed', 'error': "Unknown agent: 'hacker_agent'"}

TEST 1.5 -- Payload size limit enforced

Steps:

```
python -c "
from orchestrator.state import create_task
big = {'data': 'x' * 70000}
try:
    create_task('lead_gen', 'test', big)
except ValueError as e:
    print('BLOCKED:', e)
"
```

Expected: prints BLOCKED: Payload too large

TEST 1.6 -- JSON parser handles fenced markdown output

Steps:

```
python -c "
from utils.json_parser import extract_json
raw = '\`\`\`json\n{"title": "Test", "body": "Hello"}\n\`\`\`'
result = extract_json(raw)
print('title:', result.get('title'))
"
```

Expected: title: Test

TEST 1.7 -- JSON parser handles prose-wrapped output

Steps:

```
python -c "
from utils.json_parser import extract_json
raw = 'Here is the content: {"title": "Test", "score": 9} Hope that helps!'
result = extract_json(raw)
print('score:', result.get('score'))
"
```

Expected: score: 9

TEST 1.8 -- Email body sanitizer strips HTML

Steps:

```
python -c "
from utils.validators import sanitize_email_body
dirty = '<p>Hello <script>alert(1)</script> world</p>'
print(repr(sanitize_email_body(dirty)))
"
```

Expected: 'Hello world' (tags removed, no script)

TEST 1.9 -- Config loader reads from agents.yaml

Steps:

```
python -c "
from utils.config_loader import get_model, get_max_tokens
print(get_model('content'))
print(get_max_tokens('operations'))
"
```

"

Expected: claude-sonnet-4-6 / 1024

TEST 1.10 -- Agent base raises on missing API key

Steps:

Set ANTHROPIC_API_KEY= (empty) in env temporarily

python -c "

import os; os.environ['ANTHROPIC_API_KEY'] = "

from agents.base import BaseAgent

try:

b = BaseAgent('test', 'sys', 'claude-sonnet-4-6')

except EnvironmentError as e:

print('BLOCKED:', e)

"

Expected: prints BLOCKED: ANTHROPIC_API_KEY not set

SECTION 2: ORCHESTRATOR & APPROVAL GATE**TEST 2.1 -- Task is created and status tracked**

Steps:

python -c "

from orchestrator.state import init_db, create_task, get_stats

init_db()

t = create_task('lead_gen', 'test_task', {'q': 'test'})

print('task_id:', t[:8], '...')

print('stats:', get_stats())

"

Expected: task_id printed, total_tasks increments

TEST 2.2 -- Approval queued after dispatch (stub mode)

Note: Requires ANTHROPIC_API_KEY set

Steps:

python -c "

from orchestrator.state import init_db, get_pending_approvals

from orchestrator.router import dispatch

init_db()

result = dispatch('operations', 'test', {})

print('status:', result.get('status'))

print('pending:', len(get_pending_approvals()))

"

Expected: status: completed (operations has no approval gate)

TEST 2.3 -- Approval gate prevents double-execution

Steps:

python -c "

from orchestrator.state import init_db, create_task, queue_approval, resolve_approval

init_db()

tid = create_task('sales', 'test', {})

aid = queue_approval(tid, 'sales', 'send_outreach', {'subject': 'test', 'action': 'send'})

r1 = resolve_approval(aid, True)

print('first resolve:', r1['status'])

```
try:
r2 = resolve_approval(aid, True)
except ValueError as e:
print('second blocked:', e)
"
```

Expected: first resolve: approved / second blocked: already approved

TEST 2.4 -- Rejection stores feedback

Steps:

```
python -c "
from orchestrator.state import init_db, create_task, queue_approval, resolve_approval
init_db()
tid = create_task('content', 'test', {})
aid = queue_approval(tid, 'content', 'schedule_content', {'body': 'test', 'action': 'schedule'})
r = resolve_approval(aid, False, feedback='Tone too formal')
print('status:', r['status'])
print('feedback:', r['ceo_feedback'])
"
```

Expected: status: rejected / feedback: Tone too formal

SECTION 3: AGENT TESTS (requires ANTHROPIC_API_KEY)

TEST 3.1 -- Operations Agent runs and produces briefing

Steps:

```
python -c "
from orchestrator.state import init_db
from agents.operations import OperationsAgent
init_db()
agent = OperationsAgent()
result = agent.run({})
print('success:', result.success)
print('requires_approval:', result.requires_approval)
print('briefing excerpt:', result.output.get('briefing', "")[:120])
"
```

Expected:

success: True

requires_approval: False

briefing excerpt: shows PENDING APPROVALS / TODAY / METRICS / TOP PRIORITY sections

TEST 3.2 -- Content Agent produces valid JSON structure

Steps:

```
python -c "
from agents.content import ContentAgent
agent = ContentAgent()
result = agent.run({'platform': 'LinkedIn', 'content_type': 'Agent Tip', 'topic': 'buyer qualification'})
print('success:', result.success)
print('requires_approval:', result.requires_approval)
content = result.output.get('content', {})
print('has body:', bool(content.get('body')))
print('has title:', bool(content.get('title')))
print('has hashtags:', bool(content.get('hashtags')))
```

"

Expected: success: True, requires_approval: True, all fields present

TEST 3.3 -- Lead Gen Agent returns scored leads

Steps:

```
python -c "
from agents.lead_gen import LeadGenAgent
agent = LeadGenAgent()
result = agent.run({'query': 'Find 3 real estate brokers in Texas'})
print('success:', result.success)
leads = result.preview.get('leads', [])
print('lead count:', len(leads))
if leads:
print('first lead:', leads[0].get('name'), '| score:', leads[0].get('score'))
"
```

Expected: success: True, lead count >= 1, score field present

TEST 3.4 -- Sales Agent produces sanitized email (no HTML)

Steps:

```
python -c "
from agents.sales import SalesAgent
agent = SalesAgent()
result = agent.run({'lead': {
'id': 'test-1', 'name': 'John Doe', 'title': 'Broker',
'company': 'RE/MAX', 'email': 'john@remax.com', 'notes': 'Active team lead'
}})
print('success:', result.success)
body = result.preview.get('body', "")
import re
has_html = bool(re.search(r'<[^>]+>', body))
print('body has HTML tags:', has_html)
print('subject:', result.preview.get('subject', "")[:60])
"
```

Expected: success: True, body has HTML tags: False

TEST 3.5 -- Marketing Agent returns recommendations

Steps:

```
python -c "
from agents.marketing import MarketingAgent
agent = MarketingAgent()
result = agent.run({'context': 'Weekly strategy for RE SaaS targeting agents in Texas'})
print('success:', result.success)
recs = result.preview.get('recommendations', [])
print('recommendation count:', len(recs))
if recs:
print('first rec:', recs[0].get('campaign', ""))
"
```

Expected: success: True, recommendation count >= 1

TEST 3.6 -- Product Agent sorts by priority score

Steps:

```
python -c "
```

```

from agents.product import ProductAgent
agent = ProductAgent()
result = agent.run({})
features = result.preview.get('top_features', [])
print('top features:', len(features))
scores = [f.get('priority_score', 0) for f in features]
print('scores (should be descending):', scores)

```

Expected: scores in descending order (highest priority first)

TEST 3.7 -- Agent loop aborts at MAX_TOOL_ITERATIONS

Steps:

```

python -c "
from agents.base import MAX_TOOL_ITERATIONS
print('MAX_TOOL_ITERATIONS:', MAX_TOOL_ITERATIONS)

```

Expected: MAX_TOOL_ITERATIONS: 10

SECTION 4: DASHBOARD

TEST 4.1 -- Dashboard loads at http://localhost:8501

Steps:

streamlit run dashboard/app.py

Open http://localhost:8501

Expected:

- Title: BizOS -- CEO Command Center
- 4 metric cards visible
- Sidebar shows BizOS header in green (#407E3C)
- No emoji icons -- text labels only
- Brand green applied to buttons and headings

TEST 4.2 -- Approval page renders correctly

Steps:

Navigate to "01 approvals" page

Expected:

- Two tabs: Pending / History
- If no pending: "All clear" success message
- If pending: agent label in UPPERCASE, action in Title Case

TEST 4.3 -- Agent page shows all 6 agents

Steps:

Navigate to "02 agents" page

Expected:

- 6 rows with labels: LEADS, CONTENT, SALES, MARKETING, PRODUCT, OPS
- Each shows schedule, approval requirement, last run time
- Run button present per agent

TEST 4.4 -- Manual trigger from dashboard queues approval

Steps:

On Agents page, click Run next to CONTENT agent

Wait for spinner to complete

Expected:

- Success toast: "Done -- check Approvals"
- Navigate to Approvals page: item appears with CONTENT label
- Preview shows title, hook, body, hashtags

TEST 4.5 -- Approve executes action, item leaves queue

Steps:

On Approvals page, find the CONTENT item from test 4.4

Click Approve

Expected:

- Success message appears
- Item no longer in Pending tab
- Item appears in History tab with approved_by: ceo

TEST 4.6 -- Reject with feedback stores note

Steps:

Trigger CONTENT agent again (creates new approval)

On Approvals page, type "Too salesy" in the Note field

Click Reject

Expected:

- Warning message appears
- Item moves to History with status rejected
- Feedback "Too salesy" stored

TEST 4.7 -- CRM page shows stub warning without Airtable

Steps:

Navigate to "03 crm" page

Ensure AIRTABLE_API_KEY not set in .env

Expected:

- Orange warning: "Airtable not connected -- showing demo data"
- Demo leads table visible
- Pipeline funnel chart renders

TEST 4.8 -- Analytics page renders charts after agent activity

Steps:

Trigger at least one agent (any)

Navigate to "05 analytics" page

Expected:

- 4 KPI metrics at top
- Bar chart: Activity by Agent
- Line chart: Daily Activity
- Log table with Agent / Action / By / Time columns

TEST 4.9 -- Content page generates and previews draft

Steps:

Navigate to "04 content" page

Select LinkedIn / Market Update / topic: "interest rates"

Click Generate Content

Expected:

- Spinner shows "researching and writing"
- Success: "Draft ready -- review it in Approvals"
- Preview section shows title, hook, draft body, hashtags

TEST 4.10 -- Sidebar approval badge shows count

Steps:

Ensure at least one pending approval exists

Navigate to any page

Expected:

- Red badge in sidebar showing "! N PENDING"
- Count matches actual pending_approvals

SECTION 5: SECURITY CHECKS**TEST 5.1 -- No secrets in committed files**

Steps:

git diff HEAD --name-only

git status

Expected: .env not in tracked files, .gitignore contains .env

TEST 5.2 -- Apollo API key in header not body

File: agents/lead_gen.py line ~68

Check: headers={"X-Api-Key": api_key} not json={"api_key": ...}

Expected: key sent as header (safer than request body)

TEST 5.3 -- Email body has no HTML after sanitization

Covered by TEST 3.4 above

TEST 5.4 -- Approval cannot be executed twice

Covered by TEST 2.3 above

TEST 5.5 -- Unknown agent name rejected before DB write

Covered by TEST 1.4 above

TEST 5.6 -- Feedback length capped at 1000 chars

Steps:

```
python -c "
```

```
from orchestrator.state import init_db, create_task, queue_approval, resolve_approval
```

```
init_db()
```

```
tid = create_task('sales', 'test', {})
```

```
aid = queue_approval(tid, 'sales', 'test', {'action': 'test'})
```

```
r = resolve_approval(aid, False, feedback='x' * 5000)
```

```
print('stored feedback length:', len(r.get('ceo_feedback', '')))
```

```
"
```

Expected: stored feedback length: 1000

SECTION 6: INTEGRATION SMOKE TESTS**TEST 6.1 -- Lead Gen ? Sales handoff path**

Steps:

Run Lead Gen agent (creates leads in preview)

Approve the lead_gen approval (adds to CRM)

Run Sales agent (should pull from CRM)

Expected: Sales agent generates email for a lead from CRM

TEST 6.2 -- Slack notification stub works without key

Steps:

Ensure SLACK_WEBHOOK_URL not set

Run Operations agent

Expected:

- Agent completes without error
- Console shows: [Slack stub] *BizOS Daily Briefing*...

TEST 6.3 -- Airtable stub returns demo data

Steps:

Ensure AIRTABLE_API_KEY not set

```
python -c "
from tools.airtable_crm import get_crm_stats, get_uncontacted_leads
print(get_crm_stats())
print(len(get_uncontacted_leads()), 'leads')
"
```

Expected: returns stub data with stub: True flag

TEST 6.4 -- Full dispatch ? approve ? execute cycle

Steps:

```
python -c "
from orchestrator.state import init_db
from orchestrator.router import dispatch, execute_approved
from orchestrator.state import resolve_approval, get_pending_approvals
init_db()

# Dispatch
r = dispatch('operations', 'manual_trigger', {})
print('dispatch status:', r.get('status'))

# Operations is auto (no approval needed)
# Try content for approval cycle:
r2 = dispatch('content', 'manual_trigger', {'platform': 'LinkedIn', 'content_type': 'Agent Tip'})
if r2.get('status') == 'awaiting_approval':
    aid = r2['approval_id']
    from orchestrator.state import resolve_approval
    resolve_approval(aid, True)
    exec_result = execute_approved(aid)
    print('execute result:', exec_result)
"
```

Expected: dispatch status: completed (ops), then execute result shows scheduled: True

END OF TEST DOCUMENT

All tests passing = system ready for API key integration.

Next step: add keys to .env one at a time and re-run relevant tests.